Redes neuronales para clasificación

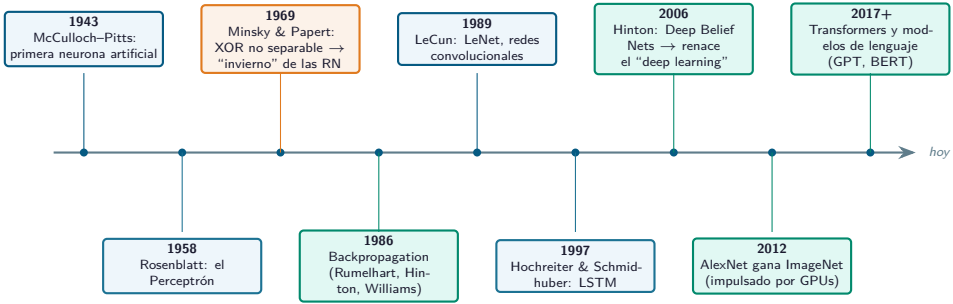
Sesión 11 · Módulo 3: Modelos de clasificación

Del perceptrón al MLP · Funciones de activación · Backpropagation

`MLPClassifier` de scikit-learn · *early stopping*

Fashion-MNIST · scikit-learn

Breve cronología de las redes neuronales



Hoy nos enfocamos en el tramo **1943–1986**: del perceptrón al MLP entrenado con backpropagation — la base sobre la que se construyó todo lo que vino después (en naranja, el "invierno" que casi detuvo el campo; en verde, los grandes puntos de inflexión).

Plan de la sesión (≈90 min)

Qué veremos

- **A.** Del **perceptrón** al **perceptrón multicapa (MLP)**: por qué una sola neurona no basta y cómo apilar capas lo resuelve
- **B. Funciones de activación**: sigmoide, ReLU y softmax — cuál usar y dónde
- **C. Backpropagation**: cómo aprende una red usando la regla de la cadena
- **D.** Entrenamiento práctico con `MLPClassifier` de scikit-learn y **early stopping**
- **Actividad.** Un clasificador MLP sobre un subconjunto de **Fashion-MNIST**

Una sola idea de fondo: una red neuronal es, en el fondo, una cadena de transformaciones simples (suma ponderada + no linealidad) que — combinadas y entrenadas con descenso de gradiente — aproximan funciones muy complejas.

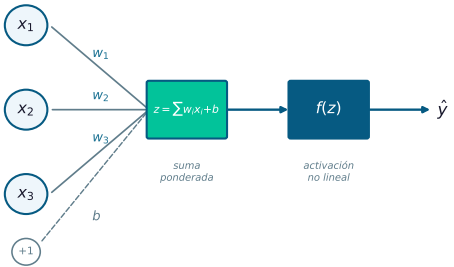
A1 · El perceptrón: la neurona más simple

La idea (Rosenblatt, 1958)

Una neurona artificial recibe entradas $x_1, \ldots, x_n$, las combina con pesos w_i y un sesgo b, y pasa el resultado por una función de activación f:

$$z = \sum_{i=1}^{n} w_i x_i + b, \qquad \hat{y} = f(z)$$

Originalmente f era una función escalón: $\hat{y} = 1$ si $z > 0$, si no $\hat{y} = 0$.



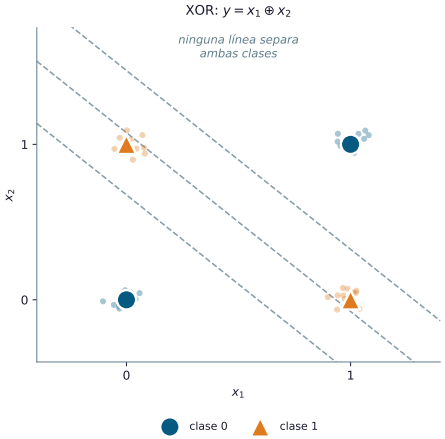
A2 · El límite del perceptrón: XOR no es separable

El problema (Minsky y Papert, 1969)

Un perceptrón de una sola capa sólo puede separar clases con una **frontera lineal** (un hiperplano).

La función XOR ($y = x_1 \oplus x_2$) **no** es linealmente separable: ninguna línea recta separa sus dos clases.

Este resultado frenó la investigación en redes neuronales durante casi dos décadas.

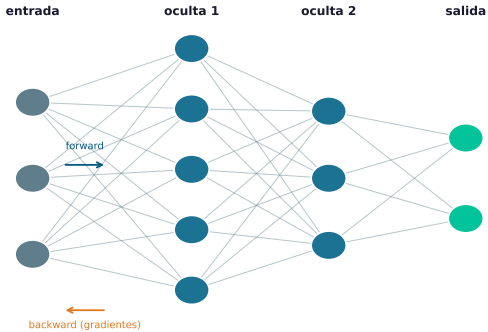


A3 · La solución: apilar capas (arquitectura MLP)

Perceptrón multicapa (*Multi-Layer Perceptron*, MLP)

Varias capas de neuronas conectadas **una tras otra**:

- **Capa de entrada**: una neurona por variable/*feature*, sin cómputo.
- **Capa(s) oculta(s)**: cada neurona combina *todas* las salidas de la capa anterior y aplica una activación no lineal.
- **Capa de salida**: produce la predicción final (una probabilidad, una clase).



A4 · Forward pass: cómo fluye la información

Capa por capa, de la entrada a la salida

Si $\mathbf{a}^{(0)} = \mathbf{x}$ es la entrada, cada capa l calcula:

$$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = f\big(\mathbf{z}^{(l)}\big)$$

donde $W^{(l)}$ es la matriz de pesos de la capa l (una fila por neurona de esa capa, una columna por neurona de la capa anterior) y f es la activación.

Ejemplo de tamaños

Entrada con 20 variables, una capa oculta de 64 neuronas y salida de 3 clases: $W^{(1)}$ es 64×20 (1 280 pesos + 64 sesgos), $W^{(2)}$ es 3×64 (192 pesos + 3 sesgos).

El *forward pass* es exactamente lo que ocurre al llamar a `.predict(X)` una vez que la red ya está entrenada.

A5 · ¿Por qué hace falta la no linealidad?

Sin activación no lineal, apilar capas no sirve de nada

Si f fuera la identidad, dos capas lineales se colapsan en una sola:

$$\mathbf{a}^{(2)} = W^{(2)}\left(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right) + \mathbf{b}^{(2)} = \underbrace{W^{(2)}W^{(1)}}_{W'}\mathbf{x} + \mathbf{b}'$$

La conclusión

Una MLP de 10 capas **sin** no linealidad equivale, matemáticamente, a un único perceptrón: $W' = W^{(10)} \cdots W^{(1)}$ sigue siendo una transformación lineal. La función de activación entre capas es lo que le da a la red la capacidad de aproximar fronteras de decisión curvas y arbitrariamente complejas (como la de XOR).

A6 · ¿Qué tan poderosa es una MLP? El teorema de aproximación universal

Cybenko (1989)

Una red con **una sola** capa oculta y activación sigmoide puede aproximar, con la precisión que se desee, **cualquier función continua** en un dominio acotado.

En la práctica: una MLP no es una arquitectura arbitraria — en principio basta para representar casi cualquier relación entrada-salida que nos interese modelar.

Hornik (1991)

Generalizó el resultado: **no es la sigmoide** la que hace posible la aproximación, sino la arquitectura multicapa en sí — se cumple para una familia amplia de funciones de activación.

En la práctica: elegir la activación (sección B) es sobre todo una cuestión de **qué tan fácil es entrenar** la red, no de si la red *puede* representar lo que buscamos.

A7 · Lo que el teorema no dice: cuántas neuronas y por qué profundidad

Barron (1993)

El resultado clásico dice que *es posible* aproximar, pero no cuántas neuronas hacen falta. Barron mostró que, para funciones razonablemente suaves, ese número **no crece exponencialmente** con la dimensión de entrada.

En la práctica: es la base teórica de por qué las redes manejan bien datos de alta dimensión (imágenes, texto) sin sufrir la **maldición de la dimensionalidad**.

Yarotsky (2017)

Comparó redes **profundas** contra redes **anchas de una sola capa**: para el mismo error, una red profunda necesita **exponencialmente menos** neuronas en total.

En la práctica: justifica por qué se prefieren redes profundas y angostas sobre una sola capa gigantesca.

De Ryck, Lanthaler & Mishra (2021)

Dieron cotas **explícitas** (números concretos, no solo garantías "en el límite") para redes con activación tanh.

En la práctica: las garantías de aproximación universal pueden hacerse **precisas y cuantificables**, útil donde se necesita certeza de precisión (p.ej. redes informadas por física).

B1 · Funciones de activación: por qué importan

El papel de f

Determina **qué tan expresiva** es la red y **qué tan fácil es entrenarla** (el gradiente de f es la pieza clave de backpropagation, sección C).

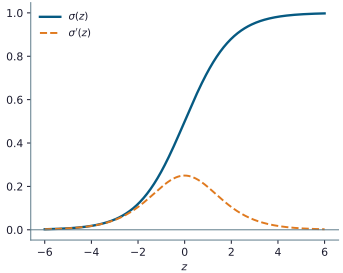
- **Capas ocultas**: necesitan una activación no lineal con gradiente bien comportado $\rightarrow$ hoy casi siempre **ReLU**.
- **Capa de salida**: depende de la tarea $\rightarrow$ **sigmoide** (binaria) o **softmax** (multiclase).

B2 · Sigmoide

Definición

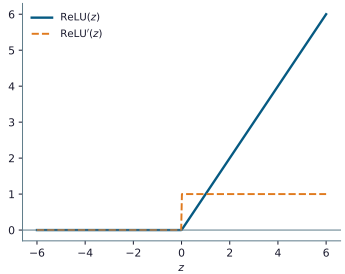
$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

- Históricamente la primera activación suave usada.
- Natural para salidas **binarias**: se interpreta como probabilidad.
- **Problema**: para $|z|$ grande, $\sigma'(z) \approx 0$ — el **gradiente se desvanece** y el aprendizaje se frena (*vanishing gradient*).



Definición

$$\mathrm{ReLU}(z) = \max(0, z)$$



- Derivada constante (1) para $z > 0 \Rightarrow$ no se desvanece el gradiente ahí $\Rightarrow$ entrenamiento mucho más rápido y estable.
- Muy barata de calcular.
- **Problema**: si una neurona cae en $z < 0$ de forma persistente, su gradiente es siempre 0 y "muere" (*dying ReLU*).

Es la activación **por defecto** para capas ocultas en `MLPClassifier` (`activation="relu"`).

B4 · Softmax: para la capa de salida multiclase

Definición

Para K clases, con $\mathbf{z} = (z_1, \ldots, z_K)$ las salidas de la última capa:

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

- Convierte K valores cualesquiera en K números **no negativos que suman 1**: una distribución de probabilidad sobre las clases.
- Se usa junto con la pérdida de **entropía cruzada categórica** (*categorical cross-entropy*).
- Para Fashion-MNIST (10 clases de prendas), la capa de salida de la MLP tiene 10 neuronas con softmax.

En `MLPClassifier`, softmax en la salida se activa automáticamente cuando hay más de dos clases — no se configura a mano.

B5 · ¿Qué activación usar dónde?

	Dónde	Uso típico
ReLU	capas ocultas	opción por defecto, casi siempre
tanh / logística	capas ocultas (menos común hoy)	redes pequeñas, capas recurrentes clásicas
Sigmoide	salida, clasificación **binaria**	probabilidad de la clase positiva
Softmax	salida, clasificación **multiclase**	probabilidad de cada una de K clases

Regla práctica: ReLU por dentro, sigmoide/softmax (según el número de clases) en la última capa.

B6 · Más allá de ReLU: funciones de activación propuestas hoy

Función	Fórmula	Rango	Nota práctica
Sigmoide	$1/(1+e^{-z})$	$(0, 1)$	la clásica; satura y desvanece el gradiente
Tanh	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	$(-1, 1)$	centrada en 0, pero aún satura
ReLU	$\max(0, z)$	$[0, \infty)$	opción por defecto; neuronas pueden "morir"
Leaky ReLU	z si $z>0$; $0.01z$ si $z\leq 0$	$(-\infty, \infty)$	evita que la neurona muera del todo
ELU	z si $z>0$; $\alpha(e^{z}-1)$ si $z\leq 0$	$(-\alpha, \infty)$	transición suave; acelera convergencia
GELU	$z\,\Phi(z)$	$(-\infty, \infty)$	estándar en Transformers (BERT, GPT)
Swish / SiLU	$z\,\sigma(z)$	$(-\infty, \infty)$	propuesta por Google; mejora ReLU en redes muy profundas
Softplus	$\ln(1+e^{z})$	$(0, \infty)$	versión suave y derivable de ReLU
Softmax	$e^{z}i \,/\, \sum_{j} e^{z}j$	$(0, 1)$, suma 1	salida multiclase (vista en B4)

En la práctica: ReLU sigue siendo el punto de partida razonable; GELU/Swish aparecen sobre todo en arquitecturas grandes (Transformers) donde una ganancia pequeña por neurona se multiplica por miles de millones de parámetros.

C1 · ¿Cómo aprende una red? (la idea general)

El objetivo

Ajustar todos los pesos $W^{(l)}$ y sesgos $\mathbf{b}^{(l)}$ de la red para **minimizar** una función de pérdida L (p. ej. entropía cruzada) que mide qué tan mal predice la red.

- Se usa **descenso de gradiente**: hay que calcular $\partial L/\partial W^{(l)}$ para **cada** peso de **cada** capa.
- Una MLP puede tener miles de pesos — calcular cada derivada por separado sería carísimo.
- **Backpropagation** es el algoritmo que calcula **todos** esos gradientes de forma eficiente, propagando el error de la salida hacia atrás, capa por capa.

C2 · La regla de la cadena en una red

Idea clave

La pérdida L depende de $W^{(1)}$ sólo **a través** de $\mathbf{z}^{(1)}$, que a su vez afecta $\mathbf{a}^{(1)}$, que afecta $\mathbf{z}^{(2)}$, ..., hasta la salida. Es una cadena de funciones compuestas, así que su derivada se calcula con la **regla de la cadena**:

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial \mathbf{a}^{(\text{salida})}} \cdot \frac{\partial \mathbf{a}^{(\text{salida})}}{\partial \mathbf{z}^{(\text{salida})}} \cdots \frac{\partial \mathbf{a}^{(1)}}{\partial \mathbf{z}^{(1)}} \cdot \frac{\partial \mathbf{z}^{(1)}}{\partial W^{(1)}}$$

"Backpropagation" = aplicar esta regla de la cadena de forma sistemática, de la última capa a la primera, reutilizando cálculos intermedios.

C3 · Ejemplo concreto: una red con dos capas ocultas

La red

$$\mathbf{x} \to \underbrace{\mathbf{z}^{(1)}{=}W^{(1)}\mathbf{x}{+}\mathbf{b}^{(1)},\ \mathbf{a}^{(1)}{=}f(\mathbf{z}^{(1)})}_{\text{capa oculta 1}} \to \underbrace{\mathbf{z}^{(2)}{=}W^{(2)}\mathbf{a}^{(1)}{+}\mathbf{b}^{(2)},\ \mathbf{a}^{(2)}{=}f(\mathbf{z}^{(2)})}_{\text{capa oculta 2}}$$

$$\to \underbrace{\mathbf{z}^{(3)}{=}W^{(3)}\mathbf{a}^{(2)}{+}\mathbf{b}^{(3)},\ \hat{y}{=}g(\mathbf{z}^{(3)})}_{\text{salida}} \to L(\hat{y}, y)$$

Las tres ecuaciones completas (regla de la cadena, sin atajos)

$$\frac{\partial L}{\partial W^{(3)}} = \frac{\partial L}{\partial \hat{y}}\,\frac{\partial \hat{y}}{\partial \mathbf{z}^{(3)}}\,\frac{\partial \mathbf{z}^{(3)}}{\partial W^{(3)}}$$

$$\frac{\partial L}{\partial W^{(2)}} = \frac{\partial L}{\partial \hat{y}}\,\frac{\partial \hat{y}}{\partial \mathbf{z}^{(3)}}\,\frac{\partial \mathbf{z}^{(3)}}{\partial \mathbf{a}^{(2)}}\,\frac{\partial \mathbf{a}^{(2)}}{\partial \mathbf{z}^{(2)}}\,\frac{\partial \mathbf{z}^{(2)}}{\partial W^{(2)}}$$

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial \hat{y}}\,\frac{\partial \hat{y}}{\partial \mathbf{z}^{(3)}}\,\frac{\partial \mathbf{z}^{(3)}}{\partial \mathbf{a}^{(2)}}\,\frac{\partial \mathbf{a}^{(2)}}{\partial \mathbf{z}^{(2)}}\,\frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{a}^{(1)}}\,\frac{\partial \mathbf{a}^{(1)}}{\partial \mathbf{z}^{(1)}}\,\frac{\partial \mathbf{z}^{(1)}}{\partial W^{(1)}}$$

Cada ecuación reutiliza exactamente los factores de la anterior — esa es la pista de por qué conviene calcular de atrás hacia adelante (C7).

C4 · Backpropagation paso a paso (ejemplo pequeño)

Una red mínima: $x \to z_1{=}wx{+}b \to a_1{=}\mathrm{ReLU}(z_1) \to \hat{y}{=}a_1 \to L{=}\frac{1}{2}(\hat{y}-y)^2$

Con $x = 2,\ w = 0.5,\ b = 0,\ y = 3$:

1. **Forward**: $z_1 = 0.5(2) = 1$; $a_1 = \mathrm{ReLU}(1) = 1 = \hat{y}$; $L = \frac{1}{2}(1-3)^2 = 2$.
2. **Backward**, paso 1: $\dfrac{\partial L}{\partial \hat{y}} = \hat{y} - y = 1 - 3 = -2$.
3. **Backward**, paso 2: $\dfrac{\partial a_1}{\partial z_1} = \mathrm{ReLU}'(1) = 1 \Rightarrow \dfrac{\partial L}{\partial z_1} = -2 \cdot 1 = -2$.
4. **Backward**, paso 3: $\dfrac{\partial z_1}{\partial w} = x = 2 \Rightarrow \dfrac{\partial L}{\partial w} = -2 \cdot 2 = -4$.

Cada paso sólo multiplica por la derivada "local" de esa operación — así se construye el gradiente completo sin recalcular todo desde cero.

C5 · Backpropagation: fórmulas generales

Para una red de L capas (capa de salida $= L$)

$$\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}} L \odot f'\left(\mathbf{z}^{(L)}\right) \qquad \text{(error en la capa de salida)}$$

$$\boldsymbol{\delta}^{(l)} = \left(W^{(l+1)}\right)^{\top} \boldsymbol{\delta}^{(l+1)} \odot f'\left(\mathbf{z}^{(l)}\right) \qquad \text{(error propagado hacia atrás)}$$

$$\frac{\partial L}{\partial W^{(l)}} = \boldsymbol{\delta}^{(l)} \left(\mathbf{a}^{(l-1)}\right)^{\top}, \qquad \frac{\partial L}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}$$

$\odot$ es el producto elemento a elemento. Cada $\boldsymbol{\delta}^{(l)}$ ("error" de la capa l) se calcula a partir del $\boldsymbol{\delta}^{(l+1)}$ de la capa siguiente — de ahí el nombre "propagación hacia atrás".

Una vez calculados todos los gradientes

Cada peso se actualiza en la dirección que **reduce** la pérdida:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}, \qquad \mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial L}{\partial \mathbf{b}^{(l)}}$$

donde η es la **tasa de aprendizaje** (*learning rate*).

- En la práctica no se usa todo el conjunto a la vez: se procesa por **mini-lotes** (*mini-batch SGD*) — más rápido y con mejor capacidad de generalización.
- Una **época** = una pasada completa por todos los datos de entrenamiento.
- η demasiado grande $\Rightarrow$ diverge; demasiado pequeño $\Rightarrow$ converge muy lento.

C7 · ¿Por qué backpropagation es eficiente?

La alternativa ingenua sería carísima

Calcular cada $\partial L/\partial w_i$ por separado, perturbando un peso a la vez y recalculando toda la red, cuesta $O(\text{número de pesos})$ pasadas completas por la red.

Lo que hace backpropagation

Reutiliza los $\boldsymbol{\delta}^{(l)}$ ya calculados en una capa para obtener los de la capa anterior (programación dinámica). El costo total es $O(1)$ pasadas hacia adelante $+$ $O(1)$ pasada hacia atrás, **sin importar cuántos pesos tenga la red**.

Esta eficiencia es lo que hizo viable entrenar redes con millones de parámetros — es, junto con la regla de la cadena, la idea que revivió a las redes neuronales en los años 80.

D1 · Entrenando una MLP en scikit-learn

```python
from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(
    hidden_layer_sizes=(128, 64),   # dos capas ocultas
    activation="relu",
    solver="adam",
    alpha=1e-3,                     # regularizacion L2
    max_iter=300,
    random_state=0)
mlp.fit(Xtr, ytr)
mlp.predict(Xte)
```

Internamente, `.fit()` ejecuta forward pass → backpropagation → actualización de pesos, repetido por `max_iter` épocas (o hasta converger).

D2 · Hiperparámetros clave de MLPClassifier

Parámetro	Qué controla
`hidden_layer_sizes`	número de capas ocultas y neuronas por capa, p. ej. `(128,64)`
`activation`	`"relu"` (por defecto), `"tanh"`, `"logistic"`
`solver`	algoritmo de optimización: `"adam"` (por defecto, recomendado), `"sgd"`, `"lbfgs"` (datos pequeños)
`alpha`	fuerza de la regularización L2 sobre los pesos (evita sobreajuste)
`learning_rate_init`	tasa de aprendizaje inicial η para `adam`/`sgd`
`batch_size`	tamaño del mini-lote en cada paso de descenso de gradiente
`max_iter`	número máximo de épocas de entrenamiento

D3 · Optimizadores en `MLPClassifier`: ¿qué `solver` elegir?

Solver	Ventajas	Desventajas	Úsalo cuando
`lbfgs`	converge en pocas iteraciones; no hay que ajustar tasa de aprendizaje	usa todo el dataset a la vez (mucha memoria); no admite mini-lotes ni `early_stopping`	dataset **pequeño** (cientos a pocos miles de filas)
`sgd`	control fino (`momentum`, `nesterovs_momentum`, calendarios de tasa de aprendizaje); escala a datasets grandes	requiere ajustar manualmente tasa de aprendizaje y momentum para converger bien	quieres control manual sobre la optimización o replicar SGD clásico
`adam`	tasa de aprendizaje adaptativa por parámetro; funciona bien con poco ajuste	algo más de memoria (guarda momentos de $1^{er}/2^{do}$ orden); a veces generaliza un poco peor que un SGD muy bien ajustado	**opción por defecto**; la mayoría de los casos, datasets medianos/grandes

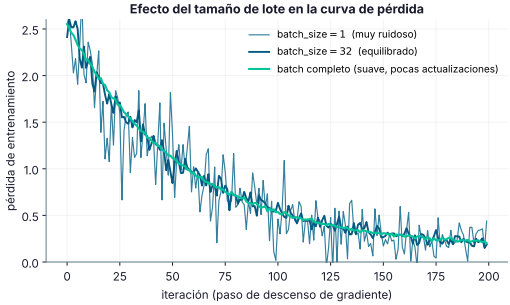
`MLPClassifier` usa `solver="adam"` por defecto. Importante: `early_stopping` (D6) sólo funciona con `"sgd"` y `"adam"` — no con `"lbfgs"`.

D4 · `batch_size`: ¿cuántos ejemplos por actualización?

Qué controla

Cuántos ejemplos de entrenamiento se usan para calcular **cada** gradiente antes de actualizar los pesos.

- **Chico** (p. ej. 1–32): gradiente ruidoso, pero **muchas** actualizaciones por época — aprende rápido al inicio.
- **Grande** / completo: gradiente preciso y curva suave, pero **pocas** actualizaciones por época — cada paso es más lento y caro.
- `MLPClassifier` usa por defecto `batch_size="auto"` $= \min(200, n)$.



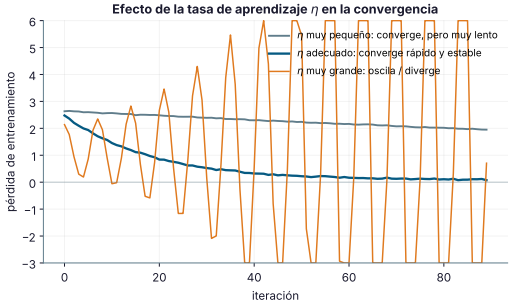
Regla práctica: lotes pequeños (32–256) suelen ser el mejor punto medio entre velocidad y estabilidad.

D5 · `learning_rate_init`: el tamaño del paso

Qué controla

Qué tan grande es cada paso de actualización:
$W \leftarrow W - \eta\, \partial L / \partial W$.

- η **muy pequeño**: converge, pero puede tardar demasiadas iteraciones.
- η **muy grande**: el paso "se pasa" del mínimo una y otra vez — oscila o diverge.
- η **adecuado**: desciende rápido y se estabiliza cerca del mínimo.

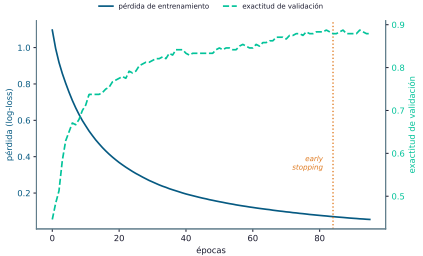


En `adam`/`sgd` se ajusta con `learning_rate_init` (valor por defecto 0.001); con `lbfgs` no aplica — el propio algoritmo decide el tamaño del paso.

D6 · Early stopping: evitar el sobreajuste

La idea

Separar una fracción de entrenamiento como **validación interna** y **detener** el entrenamiento cuando esa validación deja de mejorar — aunque la pérdida de entrenamiento siga bajando.




```
mlp = MLPClassifier(
    hidden_layer_sizes=(128, 64),
    early_stopping=True,
    validation_fraction=0.15,
    n_iter_no_change=10,
    max_iter=400)
```

`n_iter_no_change`: número de épocas sin mejora en validación antes de detener el entrenamiento.

D7 · La importancia de escalar los datos

Las MLP son sensibles a la escala de las variables

A diferencia de los árboles (Sesiones 7 y 9), una MLP combina las variables con sumas ponderadas: si una variable tiene rango $[0, 255]$ (como los píxeles de una imagen) y otra $[0, 1]$, la red aprende mucho más lento y de forma inestable.

Solución estándar

Estandarizar (media 0, varianza 1) o normalizar a $[0, 1]$ **antes** de entrenar:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler().fit(Xtr)
Xtr_s, Xte_s = scaler.transform(Xtr), scaler.transform(Xte)

# para imagenes (pixeles 0-255), tambien es comun:
X_imagenes_s = X_imagenes / 255.0
```

D8 · Código completo: MLP para Fashion-MNIST

```python
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# X: imagenes 28x28 aplanadas a 784 pixeles; y: 10 clases de prendas
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=0)

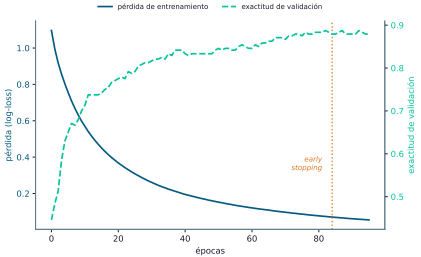
X_train_s = X_train / 255.0        # escalar pixeles a [0,1]
X_test_s = X_test / 255.0

mlp = MLPClassifier(hidden_layer_sizes=(256, 128), activation="relu",
                    solver="adam", alpha=1e-4, early_stopping=True,
                    validation_fraction=0.1, n_iter_no_change=10,
                    max_iter=200, random_state=0)
mlp.fit(X_train_s, y_train)
print(accuracy_score(y_test, mlp.predict(X_test_s)))
```

D9 · Curvas de pérdida y diagnóstico de sobreajuste

Qué revisar después de entrenar

- `mlp.loss_curve_`: pérdida de entrenamiento por época — debe bajar y estabilizarse.
- `mlp.validation_scores_`: exactitud en el conjunto de validación interna (sólo si `early_stopping=True`).
- Si la pérdida de entrenamiento sigue bajando pero la validación se estanca o empeora: **sobreajuste**.



`early_stopping` detiene el entrenamiento justo antes de que ese sobreajuste se agrave — conserva internamente los pesos del mejor punto de validación.

Síntesis: del perceptrón a una red que aprende

Pieza	Qué aporta
Perceptrón	una neurona: suma ponderada + activación; sólo separa clases linealmente
MLP (arquitectura)	apilar capas de neuronas resuelve problemas no lineales (p. ej. XOR)
Activaciones (ReLU, sigmoide, softmax)	dan a la red su capacidad no lineal; cada una tiene un rol distinto (ocultas vs. salida)
Backpropagation + regla de la cadena	calcula de forma eficiente el gradiente de la pérdida respecto a **cada** peso
`MLPClassifier` + early stopping	entrena la red en la práctica, evitando el sobreajuste

Actividad de laboratorio (individual)

Clasificador MLP sobre un subconjunto de Fashion-MNIST

Cuaderno de Jupyter con un subconjunto de Fashion-MNIST (imágenes de 28×28 píxeles, 10 clases de prendas de vestir). Trabaja **de forma individual**.

1. Carga el subconjunto, visualiza algunas imágenes de cada clase y escala los píxeles a $[0, 1]$.
2. Entrena un `MLPClassifier` con una arquitectura de tu elección y `early_stopping=True`; grafica `loss_curve_` y `validation_scores_`.
3. Compara al menos dos arquitecturas (p. ej. una capa oculta vs. dos) y dos valores de `alpha`; reporta exactitud de prueba para cada una.
4. Identifica, con la matriz de confusión, qué pares de prendas confunde más el modelo.
5. Responde: ¿qué cambió al desactivar el escalado de los píxeles? ¿Y al desactivar `early_stopping`?

Entregable: el cuaderno con las curvas de entrenamiento, la tabla comparativa de arquitecturas y tus respuestas a las preguntas de reflexión.

Resumen

Las ideas centrales de hoy

- Un **perceptrón** combina entradas linealmente y aplica una activación; por sí solo sólo separa clases **linealmente** (no resuelve XOR).
- Apilar neuronas en **capas** (MLP), con funciones de activación **no lineales** entre ellas (ReLU en las ocultas, sigmoide/softmax en la salida), permite aproximar fronteras de decisión arbitrariamente complejas.
- **Backpropagation** usa la **regla de la cadena** para calcular, de forma eficiente, el gradiente de la pérdida respecto a cada peso de cada capa — así se entrena la red con descenso de gradiente.
- En la práctica: escalar los datos, usar `MLPClassifier` con `solver="adam"` y activar `early_stopping` son pasos casi siempre recomendables.

Esto es apenas la puerta de entrada al aprendizaje profundo (*deep learning*): redes convolucionales, recurrentes y transformers son, en esencia, MLPs con arquitecturas especializadas para cada tipo de dato.



¿Preguntas?

¡Pasemos a la actividad de laboratorio: un MLP sobre Fashion-MNIST!
